Informe Desafío 2

Luis Andres Castillo Chicaiza
Facultad de Ingeniería
Universidad de Antioquia
Medellín, Colombia
luis.castillo1@udea.edu.co

Marcos Castrillón Velásquez
Facultad de Ingeniería
Universidad de Antioquia
Medellín, Colombia
marcos.castrillon@udea.edu.co

I. Contextualización

El desafio dos, o sea, ”UdeAWorldCup”, consiste en el desarrollo de un sistema de software para gestionar datos y simular el desarrollo de la Copa Mundial de la FIFA 2026. El mundial de fútbol real es un evento deportivo de alta complejidad en cuanto a la lógica en el que participan 48 selecciones nacionales. El objetivo es crear un programa que represente las entidades y acciones de interés de este entorno, utilizando el paradigma de Programación Orientada a Objetos (POO).

El simulador deberá gestionar el torneo, comenzando por una etapa de carga de datos que tomará las estadísticas de los equipos y fabricará de manera artificial las plantillas de 26 jugadores para cada selección. A partir de allí, el sistema deberá automatizar la conformación de 12 grupos mediante un proceso de sorteo.

Finalmente, la plataforma gestionará el calendario de juego y simulará la totalidad de los 104 partidos del torneo hasta determinar las estadísticas finales y el equipo campeón del mundo.

II. Análisis del problema

Para el desarrollo de *UdeAWorldCup 2026*, se identificaron los principales requisitos funcionales y las restricciones técnicas planteadas por el cliente, los cuales guían el diseño e implementación del sistema.

A. Requisitos funcionales

- **Gestión de datos:** El código debe ser capaz de leer la información histórica de los 48 equipos clasificados a partir de un archivo CSV proporcionado. Con estos datos, se inicializarán estadísticas simuladas para los 26 jugadores de cada selección. Además, esta información deberá actualizarse después de cada partido y almacenarse en un nuevo archivo definido por los desarrolladores.

- **Sorteos y fase de grupos:** El programa debe organizar a las selecciones en 4 bombos según el ranking de la FIFA (ubicando al país anfitrión en el bombo 1). A partir de esto, se conformarán 12 grupos de 4 equipos elegidos de forma aleatoria, asegurando que no coincidan equipos de la misma confederación.

- **Simulación de partidos:** El sistema debe simular los 104 partidos del torneo utilizando un modelo matemático basado en ”goles esperados”. Adicionalmente, en cada partido se seleccionarán aleatoriamente 11 jugadores por equipo, a quienes se les asignarán estadísticas como goles, tarjetas (amarillas y rojas) y faltas, siguiendo probabilidades previamente definidas.

- **Eliminatorias y desempates:** Una vez finalizada la fase de grupos, el sistema generará la tabla de posiciones. La clasificación a octavos de final se realizará primero por puntos y, en caso de empate, se aplicarán criterios como diferencia de goles, goles a favor y un sorteo. En las fases eliminatorias, si un partido termina en empate, se utilizará el ranking FIFA para definir al ganador, considerando además un tiempo total de juego de 120 minutos.

- **Rendimiento:** El sistema debe generar estadísticas finales del torneo, incluyendo los 4 mejores equipos, los máximos goleadores y la representación de cada confederación en las etapas finales. Además, durante la ejecución, el programa debe registrar la cantidad de iteraciones realizadas y el consumo de memoria dinámica.

B. Restricciones técnicas

- **Lenguaje:** La solución debe desarrollarse utilizando Programación Orientada a Objetos (POO), modular basado en múltiples archivos. No está permitido el uso de sintaxis propia de C.

- **Restricción de librerías estándar:** No se permite el uso de la STL, ni de *structs*, ni de librerías externas (con excepción de la librería de números aleatorios).

- **Estructuras de datos personalizadas:** Todas las colecciones de datos (como listas de jugadores o calendarios de partidos) deben implementarse mediante estructuras propias, haciendo uso de punteros y memoria

dinámica como base.

- **Sobrecarga de operadores:** Se aplicaron dos sobrecargas clave para facilitar la lógica del torneo :

  -Comparación de equipos (`operator<`): Personalizamos el operador ”menor que” en la clase `equipo`. Gracias a esto, el sistema sabe comparar dos selecciones automáticamente siguiendo las reglas de la FIFA (puntos, diferencia de gol y ranking). Esto permite que el código ordene las tablas de posiciones de forma natural, como si estuviera comparando números simples.

  -Acceso directo en listas (`operator[]`): Agregamos los corchetes a nuestra clase `list`. Esto nos permite entrar a la información de un jugador o un equipo escribiendo simplemente su posición (por ejemplo, `lista[5]`).

III. Diseño de la Arquitectura

Para que el simulador funcione de manera organizada, el código se dividió en diferentes módulos que interactúan entre sí, para guardar los datos en la memoria de los jugadores hasta la lógica global que maneja el mundial. Los archivos creados para este desafío fueron los siguientes:

A. Almacenamiento y Manejo de Datos

Luego crear la base de las clases de equipos o los jugadores, se construyó la base que permite guardar la información de forma dinámica:

- **Nodos (`nodo`):** Su función es guardar un dato y ”recordar” quién es el siguiente en la fila. Al usar plantillas, puede guardar información como un jugador, un equipo,partido etc.).

- **Listas (`list`):** En lugar de usar arreglos fijos, esta lista crece o se encoge según se necesite. Cada vez que añadimos o quitamos un equipo, la lista actualiza un contador global de memoria para que sepamos exactamente cuántos bytes está consumiendo el programa en tiempo real.

B. Componentes del Mundial

- **Jugadores (`jugador`):** Guarda la información básica de cada deportista, como su nombre,dorsal y posición. También lleva la cuenta de lo que hace en cada partido (goles, tarjetas y faltas) y lo que va acumulando en todo el torneo.

- **Equipos (`equipo`):** Un equipo es básicamente un grupo de 26 jugadores. Esta clase calcula el rendimiento colectivo y tiene una regla especial de comparación: sabe que un equipo es ”mejor” que otro si tiene más puntos, mejor diferencia de goles o un ranking FIFA más alto.

- **Partidos (`partido`):** Esta clase toma dos equipos y simula un encuentro basándose en sus estadísticas. Al terminar, se encarga de repartir los goles a los jugadores y los puntos a los equipos.

- **Grupos (`grupo`):** Reúne a cuatro equipos y se encarga de organizar los enfrentamientos entre ellos. Al final de sus partidos, genera la tabla de posiciones usando las reglas de desempate.

C. Control Central

- **UdeaWorldCup:** Es la parte del código que lee los archivos externos con la información de las selecciones, realiza el sorteo de los grupos y maneja el calendario desde el primer partido hasta la final. Al cerrar el programa, esta sección se asegura de limpiar la memoria y mostrar los resultados finales.

IV. Flujo General de Ejecución

El funcionamiento del simulador sigue una secuencia lógica que imita lo que sucede en la vida real antes y durante un mundial. El proceso se divide en cuatro etapas principales:

1) **Carga y Preparación:**Se leen el archivo CSV que contienen la información de las 48 selecciones. En este , el sistema no solo crea a los equipos, sino que genera automáticamente a los 26 jugadores de cada plantilla, asignándoles posiciones y dorsales. Al finalizar esta etapa, toda la información está organizada en una lista maestra de selecciones.

2) **Sorteo de Grupos:** Una vez tenemos los equipos, el programa ejecuta el sorteo. No es una selección al azar; el código separa a los equipos por su nivel (Ranking FIFA) y se asegura de que no queden demasiados países del mismo continente en un mismo grupo. Al terminar, tenemos 12 grupos de 4 equipos cada uno, listos para jugar.

3) **Fase de Grupos:** Aquí se juegan los primeros 72 partidos. Cada equipo se enfrenta a los otros tres de su grupo. El sistema registra los resultados, actualiza los puntos de cada selección y suma los goles a las estadísticas de los jugadores. Al cerrar esta fase, el programa revisa las tablas de posiciones y selecciona a los dos mejores de cada grupo y a los ocho mejores terceros para avanzar.

4) **Rondas de Eliminación y Cierre:** Con los 32 clasificados, el simulador genera las llaves (desde dieciseisavos hasta la final). En esta etapa, el programa no permite empates; si un partido termina igualado, avanza a la prorroga y se definen probabilidades mas altas para que anoten. El flujo termina mostrando el podio de campeones, el máximo goleador y un reporte

de cuánta memoria consumió el programa durante todo el proceso.

V. Problemas que Afrontamos

Durante el desarrollo de *UdeAWorldCup*, nos encontramos con varios retos técnicos que nos obligaron a repensar la lógica del código:

- **Sorteo:** Lograr que el sorteo cumpliera con las reglas de la FIFA fue difícil. Para ello, se creo una lógica que verificara el país de cada equipo para evitar que, por ejemplo, tres selecciones sudamericanas terminaran en el mismo grupo.

- **Criterios de desempate:** En el mundial, cuando dos equipos empatan en puntos, la regla de quién pasa se da por goles a favor, diferencia y ranking. Implementar esto en el código requirió que la clase `equipo` comparara con otros evaluando múltiples variables en orden de prioridad hasta encontrar un ganador.

- **Gestión de memoria :** Al manejar la memoria manualmente, nos dimos cuenta de que no podíamos simplemente usar `delete` con los equipos que iban siendo eliminados, ya que necesitábamos sus datos para los reportes del final. El reto lo solucionamos creando listas auxiliares de punteros. Así, en lugar de borrar información, las selecciones que avanzan de ronda simplemente se referencian en nuevas listas. De esta forma, los 48 equipos y todos sus jugadores se mantienen intactos en la memoria principal hasta el cierre del simulador, permitiéndonos consultar cualquier estadística (como el máximo goleador del torneo) sin perder datos.

- **Equilibrio en los resultados:** Si la simulación era demasiado aleatoria, los equipos débiles ganaban siempre; si la hacíamos fija, siempre ganaba el mejor ranking. El reto fue ajustar la probabilidad para que el ranking pesara, pero que siempre existiera la posibilidad de una sorpresa en el marcador.

- **Estadísticas (`equipo` y `partido`):** Un problema fue mantener la información actualizada.Cuando un jugador anota un gol, ese dato debe verse reflejado en tres partes al mismo tiempo: jugador, en el marcador del partido y en la tabla de posiciones de su equipo.Para ello, se crearon funciones para que los puntos y goles se enviaran desde el partido hacia las estadísticas globales de la selección.

GitHub: https://github.com/LuisC645/Desafio_2
YouTube: https://youtu.be/PohPpC0wAhE

VI. Diagrama de clases

A continuación se muestra el diagrama de clases presentado para este desafio:

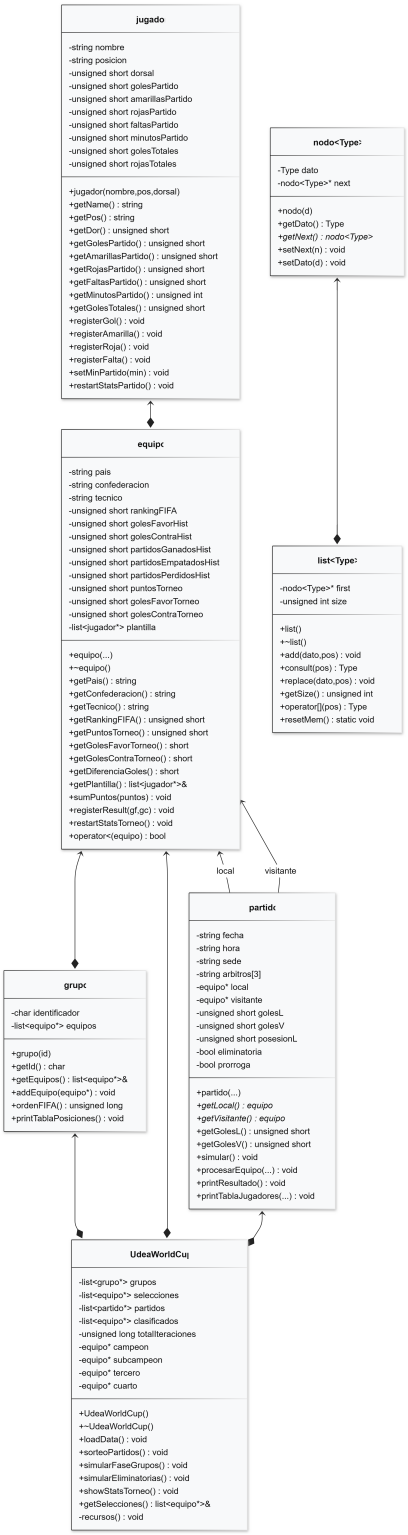

Fig. 1. Diagrama de clases